AI-ASSISTED CODING

ASSIGNMENT -19.4

NAME:V.SIDDARTHA

HALL:2303A52062

Task Description #1 (Sorting – Merge Sort Implementation)

• Task: Use AI to generate a Python program that implements the Merge Sort algorithm.

• Instructions:

o Prompt AI to create a function merge_sort(arr) that sorts a list in ascending order.

o Ask AI to include time complexity and space complexity in the function docstring.

o Verify the generated code with test cases.

• Expected Output:

o A functional Python script implementing Merge Sort with proper documentation.

PROMPT

Create a Python function merge_sort(arr) that sorts a list in ascending order using Merge Sort. Add a docstring with time and space complexity, and include test cases.

CODE

```
def merge_sort(arr):
    """
    Sort a list in ascending order using Merge Sort.

    Time Complexity:
    - Best: O(n log n)
```

```
    - Average: O(n log n)
    - Worst: O(n log n)

    Space Complexity:
    - O(n) auxiliary space for merging
    """
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return _merge(left, right)

def _merge(left, right):
    merged = []
    i = 0
    j = 0

    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            merged.append(left[i])
            i += 1
        else:
            merged.append(right[j])
            j += 1

    merged.extend(left[i:])
```

```
    merged.extend(right[j:])
    return merged

if __name__ == "__main__":
    test_cases = [
        [5, 2, 9, 1, 5, 6],
        [3, 3, 2, 1],
        [],
        [10],
        [-2, -5, 0, 7],
    ]

    for case in test_cases:
        print(f"Input: {case} -> Sorted: {merge_sort(case)}")
```

OUTPUT

```
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding> & C:\Users\sidda\AppData\L
ocal\Programs\Python\Python314\python.exe "c:/Users/sidda/OneDrive/Documents/Desktop/AI - Ass
isted coding/.github/12.5.PY/task1_merge_sort.py"
Input: [5, 2, 9, 1, 5, 6] -> Sorted: [1, 2, 5, 5, 6, 9]
Input: [3, 3, 2, 1] -> Sorted: [1, 2, 3, 3]
Input: [] -> Sorted: []
Input: [10] -> Sorted: [10]
Input: [-2, -5, 0, 7] -> Sorted: [-5, -2, 0, 7]
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding>
```

Task Description #2 (Searching – Binary Search with AI Optimization)

- **Task: Use AI to create a binary search function that finds a target element in a sorted list.**

• Instructions:

o Prompt AI to create a function binary_search(arr, target) returning the index of the target or -1 if not found.

o Include docstrings explaining best, average, and worst-case complexities.

o Test with various inputs.

• Expected Output:

o Python code implementing binary search with AI-generated comments and docstrings.

Prompt

Create a Python function binary_search(arr, target) for a sorted list. Return index if found, else -1. Include docstring with best, average, and worst complexity, and add test cases.

Code

```
ef binary_search(arr, target):
    """
    Perform binary search on a sorted list.

    Returns:
    - Index of target if found
    - -1 if target is not found

    Time Complexity:
    - Best: O(1)
    - Average: O(log n)
```

```
    - Worst: O(log n)

    Space Complexity:
    - O(1)
    """
    left = 0
    right = len(arr) - 1

    while left <= right:
        mid = (left + right) // 2

        if arr[mid] == target:
            return mid
        if arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1

    return -1

if __name__ == "__main__":
    nums = [1, 3, 5, 7, 9, 11]
    tests = [7, 1, 11, 4, 10]

    for target in tests:
        print(f"Target {target}: index {binary_search(nums, target)}")
```

Output

```
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding> & C:\Users\sidda\AppData\L
ocal\Programs\Python\Python314\python.exe "c:/Users/sidda/OneDrive/Documents/Desktop/AI - Ass
isted coding/.github/12.5.PY/task2_binary_search.py"
Target 7: index 3
Target 1: index 0
Target 11: index 5
Tar
Tar
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding>
```

Focus folder in explorer (ctrl + click)

Task Description #3: Smart Healthcare Appointment Scheduling System

A healthcare platform maintains appointment records containing appointment ID, patient name, doctor name, appointment time, and consultation fee. The system needs to:

1. Search appointments using appointment ID.

2. Sort appointments based on time or consultation fee.

Student Task

- **Use AI to recommend suitable searching and sorting algorithms.**
- **Justify the selected algorithms.**
- **Implement the algorithms in Python.**

Prompt

Recommend suitable searching and sorting algorithms for appointment records, justify the choices, and implement Python code to:

1) Search by appointment ID

2) Sort by appointment time or consultation fee

Code

```
from datetime import datetime


def merge_sort_records(records, key_func):
    if len(records) <= 1:
        return records


    mid = len(records) // 2
    left = merge_sort_records(records[:mid], key_func)
    right = merge_sort_records(records[mid:], key_func)
    return _merge(left, right, key_func)


def _merge(left, right, key_func):
    merged = []
    i = 0
    j = 0


    while i < len(left) and j < len(right):
        if key_func(left[i]) <= key_func(right[j]):
            merged.append(left[i])
            i += 1
        else:
            merged.append(right[j])
            j += 1


    merged.extend(left[i:])
    merged.extend(right[j:])
    return merged
```

```
def binary_search_by_id(records_sorted_by_id, target_id):
    left = 0
    right = len(records_sorted_by_id) - 1

    while left <= right:
        mid = (left + right) // 2
        current_id = records_sorted_by_id[mid]["appointment_id"]

        if current_id == target_id:
            return records_sorted_by_id[mid]
        if current_id < target_id:
            left = mid + 1
        else:
            right = mid - 1

    return None

if __name__ == "__main__":
    appointments = [
        {"appointment_id": 103, "patient": "Asha", "doctor": "Dr. Rao", "time": "2026-04-20
10:30", "fee": 700},
        {"appointment_id": 101, "patient": "Kiran", "doctor": "Dr. Sen", "time": "2026-04-20
09:00", "fee": 500},
        {"appointment_id": 102, "patient": "Maya", "doctor": "Dr. Roy", "time": "2026-04-20
11:00", "fee": 600},
    ]

    by_id = merge_sort_records(appointments, lambda x: x["appointment_id"])
```

```
by_time = merge_sort_records(appointments, lambda x: datetime.strptime(x["time"], "%Y-%m-%d %H:%M"))

by_fee = merge_sort_records(appointments, lambda x: x["fee"])


print("Search appointment ID 102:", binary_search_by_id(by_id, 102))

print("Sorted by time:", by_time)

print("Sorted by fee:", by_fee)
```

Output

```
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding> & C:\Users\sidda\AppData\L
ocal\Programs\Python\Python314\python.exe "c:/Users/sidda/OneDrive/Documents/Desktop/AI - Ass
isted coding/.github/12.5.PY/task3_healthcare_appointments.py"
Search appointment ID 102: {'appointment_id': 102, 'patient': 'Maya', 'doctor': 'Dr. Roy', 't
ime': '2026-04-20 11:00', 'fee': 600}
Sorted by time: [{'appointment_id': 101, 'patient': 'Kiran', 'doctor': 'Dr. Sen', 'time': '20
26-04-20 09:00', 'fee': 500}, {'appointment_id': 103, 'patient': 'Asha', 'doctor': 'Dr. Rao',
 'time': '2026-04-20 10:30', 'fee': 700}, {'appointment_id': 102, 'patient': 'Maya', 'doctor'
: 'Dr. Roy', 'time': '2026-04-20 11:00', 'fee': 600}]
Sorted by fee: [{'appointment_id': 101, 'patient': 'Kiran', 'doctor': 'Dr. Sen', 'time': '202
6-04-20 09:00', 'fee': 500}, {'appointment_id': 102, 'patient': 'Maya', 'doctor': 'Dr. Roy',
'time': '2026-04-20 11:00', 'fee': 600}, {'appointment_id': 103, 'patient': 'Asha', 'doctor':
 'Dr. Rao', 'time': '2026-04-20 10:30', 'fee': 700}]
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding>
```

Task Description #4: Railway Ticket Reservation System

Scenario

A railway reservation system stores booking details such as ticket ID, passenger name, train number, seat number, and travel date. The system must:

1. Search tickets using ticket ID.

2. Sort bookings based on travel date or seat number.

Student Task

- **Identify efficient algorithms using AI assistance.**
- **Justify the algorithm choices.**
- **Implement searching and sorting in Python.**

Prompt

Identify efficient search and sort algorithms for railway booking records, justify the selection, and implement Python code to:

1) Search by ticket ID

2) Sort by travel date or seat number

Code

```
from datetime import datetime


def merge_sort_records(records, key_func):
    if len(records) <= 1:
        return records


    mid = len(records) // 2
    left = merge_sort_records(records[:mid], key_func)
    right = merge_sort_records(records[mid:], key_func)
    return _merge(left, right, key_func)


def _merge(left, right, key_func):
    out = []
    i = 0
    j = 0


    while i < len(left) and j < len(right):
        if key_func(left[i]) <= key_func(right[j]):
            out.append(left[i])
            i += 1
```

```
        else:
            out.append(right[j])
            j += 1

    out.extend(left[i:])
    out.extend(right[j:])
    return out

def binary_search_ticket(records_sorted, ticket_id):
    left = 0
    right = len(records_sorted) - 1

    while left <= right:
        mid = (left + right) // 2
        current_id = records_sorted[mid]["ticket_id"]

        if current_id == ticket_id:
            return records_sorted[mid]
        if current_id < ticket_id:
            left = mid + 1
        else:
            right = mid - 1

    return None

if __name__ == "__main__":
    bookings = [
```

{"ticket_id": 205, "passenger": "Ravi", "train_no": "12701", "seat_no": 32, "travel_date": "2026-05-01"},

{"ticket_id": 201, "passenger": "Neha", "train_no": "12621", "seat_no": 12, "travel_date": "2026-04-29"},

{"ticket_id": 203, "passenger": "Isha", "train_no": "12810", "seat_no": 7, "travel_date": "2026-04-30"},

]

```
by_id = merge_sort_records(bookings, lambda x: x["ticket_id"])
by_date = merge_sort_records(bookings, lambda x: datetime.strptime(x["travel_date"], "%Y-%m-%d"))
by_seat = merge_sort_records(bookings, lambda x: x["seat_no"])

print("Search ticket ID 203:", binary_search_ticket(by_id, 203))
print("Sorted by travel date:", by_date)
print("Sorted by seat number:", by_seat)
```

Output

```
  Dr. Rao ,  time :  2026-04-20 10:30 ,  fee : 700}]
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding> & C:\Users\sidda\AppData\L
ocal\Programs\Python\Python314\python.exe "c:/Users/sidda/OneDrive/Documents/Desktop/AI - Ass
isted coding/.github/12.5.PY/task4_railway_reservation.py"
Search ticket ID 203: {'ticket_id': 203, 'passenger': 'Isha', 'train_no': '12810', 'seat_no':
 7, 'travel_date': '2026-04-30'}
Sorted by travel date: [{'ticket_id': 201, 'passenger': 'Neha', 'train_no': '12621', 'seat_no
': 12, 'travel_date': '2026-04-29'}, {'ticket_id': 203, 'passenger': 'Isha', 'train_no': '128
10', 'seat_no': 7, 'travel_date': '2026-04-30'}, {'ticket_id': 205, 'passenger': 'Ravi', 'tra
in_no': '12701', 'seat_no': 32, 'travel_date': '2026-05-01'}]
Sorted by seat number: [{'ticket_id': 203, 'passenger': 'Isha', 'train_no': '12810', 'seat_no
': 7, 'travel_date': '2026-04-30'}, {'ticket_id': 201, 'passenger': 'Neha', 'train_no': '1262
1', 'seat_no': 12, 'travel_date': '2026-04-29'}, {'ticket_id': 205, 'passenger': 'Ravi', 'tra
in_no': '12701', 'seat_no': 32, 'travel_date': '2026-05-01'}]
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding>
```

Task Description #5: Smart Hostel Room Allocation System

A hostel management system stores student room allocation details

including student ID, room number, floor, and allocation date. The system needs to:

1. Search allocation details using student ID.

2. Sort records based on room number or allocation date.

Student Task

- **Use AI to suggest optimized algorithms.**
- **Justify the selections.**
- **Implement the solution in Python.**

Prompt

Use AI to suggest optimized algorithms, justify the choices, and implement Python code to:

1) Search allocation details by student ID

2) Sort by room number or allocation date

Code

```
rom datetime import datetime


def merge_sort_records(records, key_func):
    if len(records) <= 1:
        return records


    mid = len(records) // 2
    left = merge_sort_records(records[:mid], key_func)
    right = merge_sort_records(records[mid:], key_func)
    return _merge(left, right, key_func)


def _merge(left, right, key_func):
    merged = []
```

```
    i = 0
    j = 0

    while i < len(left) and j < len(right):
        if key_func(left[i]) <= key_func(right[j]):
            merged.append(left[i])
            i += 1
        else:
            merged.append(right[j])
            j += 1

    merged.extend(left[i:])
    merged.extend(right[j:])
    return merged

def binary_search_student(records_sorted, student_id):
    left = 0
    right = len(records_sorted) - 1

    while left <= right:
        mid = (left + right) // 2
        current_id = records_sorted[mid]["student_id"]

        if current_id == student_id:
            return records_sorted[mid]
        if current_id < student_id:
            left = mid + 1
        else:
```

```
        right = mid - 1

    return None

if __name__ == "__main__":
    allocations = [
        {"student_id": 301, "room_no": 204, "floor": 2, "allocation_date": "2026-01-10"},
        {"student_id": 299, "room_no": 102, "floor": 1, "allocation_date": "2026-01-08"},
        {"student_id": 305, "room_no": 305, "floor": 3, "allocation_date": "2026-01-15"},
    ]

    by_id = merge_sort_records(allocations, lambda x: x["student_id"])
    by_room = merge_sort_records(allocations, lambda x: x["room_no"])
    by_date = merge_sort_records(allocations, lambda x:
datetime.strptime(x["allocation_date"], "%Y-%m-%d"))

    print("Search student ID 301:", binary_search_student(by_id, 301))
    print("Sorted by room number:", by_room)
    print("Sorted by allocation date:", by_date)
```

Output

```
PROBLEMS    DEBUG CONSOLE    TERMINAL    PORTS    POSTMAN CONSOLE

PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding> & C:\Users\sidda\AppData\L
ocal\Programs\Python\Python314\python.exe "c:/Users/sidda/OneDrive/Documents/Desktop/AI - Ass
isted coding/.github/12.5.PY/task5_hostel_allocation.py"
Search student ID 301: {'student_id': 301, 'room_no': 204, 'floor': 2, 'allocation_date': '20
26-01-10'}
Sorted by room number: [{'student_id': 299, 'room_no': 102, 'floor': 1, 'allocation_date': '2
026-01-08'}, {'student_id': 301, 'room_no': 204, 'floor': 2, 'allocation_date': '2026-01-10'}
, {'student_id': 305, 'room_no': 305, 'floor': 3, 'allocation_date': '2026-01-15'}]
Sorted by allocation date: [{'student_id': 299, 'room_no': 102, 'floor': 1, 'allocation_date'
: '2026-01-08'}, {'student_id': 301, 'room_no': 204, 'floor': 2, 'allocation_date': '2026-01-
10'}, {'student_id': 305, 'room_no': 305, 'floor': 3, 'allocation_date': '2026-01-15'}]
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding>
```

Task Description #6: Online Movie Streaming Platform

A streaming service maintains movie records with movie ID, title,

genre, rating, and release year. The platform needs to:

1. Search movies by movie ID.

2. Sort movies based on rating or release year.

Student Task

- **Recommend searching and sorting algorithms using AI.**
- **Justify the chosen algorithms.**
- **Implement Python functions.**

Prompt

Recommend searching and sorting algorithms, justify them, and implement

Python code to:

1) Search movie records by movie ID

2) Sort by rating or release year

Code

```
def merge_sort_records(records, key_func):
    if len(records) <= 1:
```

```
        return records

    mid = len(records) // 2
    left = merge_sort_records(records[:mid], key_func)
    right = merge_sort_records(records[mid:], key_func)
    return _merge(left, right, key_func)

def _merge(left, right, key_func):
    out = []
    i = 0
    j = 0

    while i < len(left) and j < len(right):
        if key_func(left[i]) <= key_func(right[j]):
            out.append(left[i])
            i += 1
        else:
            out.append(right[j])
            j += 1

    out.extend(left[i:])
    out.extend(right[j:])
    return out

def binary_search_movie(records_sorted, movie_id):
    left = 0
    right = len(records_sorted) - 1
```

```
    while left <= right:
        mid = (left + right) // 2
        current_id = records_sorted[mid]["movie_id"]

        if current_id == movie_id:
            return records_sorted[mid]
        if current_id < movie_id:
            left = mid + 1
        else:
            right = mid - 1

    return None

if __name__ == "__main__":
    movies = [
        {"movie_id": 410, "title": "Skyline", "genre": "Sci-Fi", "rating": 8.3, "release_year":
2023},
        {"movie_id": 405, "title": "River Song", "genre": "Drama", "rating": 7.5,
"release_year": 2020},
        {"movie_id": 420, "title": "Night Ops", "genre": "Action", "rating": 8.9,
"release_year": 2024},
    ]

    by_id = merge_sort_records(movies, lambda x: x["movie_id"])
    by_rating = merge_sort_records(movies, lambda x: x["rating"])
    by_year = merge_sort_records(movies, lambda x: x["release_year"])

    print("Search movie ID 420:", binary_search_movie(by_id, 420))
    print("Sorted by rating:", by_rating)
```

```
print("Sorted by release year:", by_year)
```

Output

```
10'}, {'student_id': 305, 'room_no': 305, 'floor': 3, 'allocation_date': '2026-01-15'}]
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding> & C:\Users\sidda\AppData\L
ocal\Programs\Python\Python314\python.exe "c:/Users/sidda/OneDrive/Documents/Desktop/AI - Ass
isted coding/.github/12.5.PY/task6_movie_streaming.py"
Search movie ID 420: {'movie_id': 420, 'title': 'Night Ops', 'genre': 'Action', 'rating': 8.9
, 'release_year': 2024}
Sorted by rating: [{'movie_id': 405, 'title': 'River Song', 'genre': 'Drama', 'rating': 7.5,
'release_year': 2020}, {'movie_id': 410, 'title': 'Skyline', 'genre': 'Sci-Fi', 'rating': 8.3
, 'release_year': 2023}, {'movie_id': 420, 'title': 'Night Ops', 'genre': 'Action', 'rating':
 8.9, 'release_year': 2024}]
Sorted by release year: [{'movie_id': 405, 'title': 'River Song', 'genre': 'Drama', 'rating':
 7.5, 'release_year': 2020}, {'movie_id': 410, 'title': 'Skyline', 'genre': 'Sci-Fi', 'rating
': 8.3, 'release_year': 2023}, {'movie_id': 420, 'title': 'Night Ops', 'genre': 'Action', 'ra
ting': 8.9, 'release_year': 2024}]
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding>
```

Task Description #7: Smart Agriculture Crop Monitoring System

An agriculture monitoring system stores crop data with crop ID, crop name, soil moisture level, temperature, and yield estimate. Farmers need to:

1. Search crop details using crop ID.

2. Sort crops based on moisture level or yield estimate.

Student Task

- **Use AI-assisted reasoning to select algorithms.**
- **Justify algorithm suitability.**
- **Implement searching and sorting in Python.**

Prompt

Use AI-assisted reasoning to select search and sort algorithms, justify the choices, and implement Python code to:

1) Search crop details by crop ID

2) Sort by moisture level or yield estimate

Code

```
def merge_sort_records(records, key_func):
    if len(records) <= 1:
        return records

    mid = len(records) // 2
    left = merge_sort_records(records[:mid], key_func)
    right = merge_sort_records(records[mid:], key_func)
    return _merge(left, right, key_func)

def _merge(left, right, key_func):
    merged = []
    i = 0
    j = 0

    while i < len(left) and j < len(right):
        if key_func(left[i]) <= key_func(right[j]):
            merged.append(left[i])
            i += 1
        else:
            merged.append(right[j])
            j += 1

    merged.extend(left[i:])
    merged.extend(right[j:])
    return merged

def binary_search_crop(records_sorted, crop_id):
```

```
    left = 0
    right = len(records_sorted) - 1

    while left <= right:
        mid = (left + right) // 2
        current_id = records_sorted[mid]["crop_id"]

        if current_id == crop_id:
            return records_sorted[mid]
        if current_id < crop_id:
            left = mid + 1
        else:
            right = mid - 1

    return None

if __name__ == "__main__":
    crops = [
        {"crop_id": 501, "crop_name": "Wheat", "moisture": 32.5, "temperature": 28.0,
"yield_estimate": 4.2},
        {"crop_id": 498, "crop_name": "Rice", "moisture": 45.0, "temperature": 30.5,
"yield_estimate": 5.1},
        {"crop_id": 507, "crop_name": "Maize", "moisture": 29.0, "temperature": 27.0,
"yield_estimate": 3.8},
    ]

    by_id = merge_sort_records(crops, lambda x: x["crop_id"])
    by_moisture = merge_sort_records(crops, lambda x: x["moisture"])
    by_yield = merge_sort_records(crops, lambda x: x["yield_estimate"])
```

```
print("Search crop ID 501:", binary_search_crop(by_id, 501))
print("Sorted by moisture level:", by_moisture)
print("Sorted by yield estimate:", by_yield)
```

Output

```
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding> & C:\Users\sidda\AppData\L
ocal\Programs\Python\Python314\python.exe "c:/Users/sidda/OneDrive/Documents/Desktop/AI - Ass
isted coding/.github/12.5.PY/task7_agriculture_monitoring.py"
Search crop ID 501: {'crop_id': 501, 'crop_name': 'Wheat', 'moisture': 32.5, 'temperature': 2
8.0, 'yield_estimate': 4.2}
Sorted by moisture level: [{'crop_id': 507, 'crop_name': 'Maize', 'moisture': 29.0, 'temperat
ure': 27.0, 'yield_estimate': 3.8}, {'crop_id': 501, 'crop_name': 'Wheat', 'moisture': 32.5,
'temperature': 28.0, 'yield_estimate': 4.2}, {'crop_id': 498, 'crop_name': 'Rice', 'moisture'
: 45.0, 'temperature': 30.5, 'yield_estimate': 5.1}]
Sorted by yield estimate: [{'crop_id': 507, 'crop_name': 'Maize', 'moisture': 29.0, 'temperat
ure': 27.0, 'yield_estimate': 3.8}, {'crop_id': 501, 'crop_name': 'Wheat', 'moisture': 32.5,
'temperature': 28.0, 'yield_estimate': 4.2}, {'crop_id': 498, 'crop_name': 'Rice', 'moisture'
: 45.0, 'temperature': 30.5, 'yield_estimate': 5.1}]
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding>
```

Python
Python

Task Description #8: Airport Flight Management System

An airport system stores flight information including flight ID, airline name, departure time, arrival time, and status. The system must:

1. Search flight details using flight ID.

2. Sort flights based on departure time or arrival time.

Student Task

- **Use AI to recommend algorithms.**
- **Justify the algorithm selection.**
- **Implement searching and sorting logic in Python.**

Prompt

Recommend search and sort algorithms, justify the choices, and implement Python code to:

1) Search flights by flight ID

2) Sort by departure time or arrival time

Code

```
from datetime import datetime


def merge_sort_records(records, key_func):
    if len(records) <= 1:
        return records

    mid = len(records) // 2
    left = merge_sort_records(records[:mid], key_func)
    right = merge_sort_records(records[mid:], key_func)
    return _merge(left, right, key_func)

def _merge(left, right, key_func):
    merged = []
    i = 0
    j = 0

    while i < len(left) and j < len(right):
        if key_func(left[i]) <= key_func(right[j]):
            merged.append(left[i])
            i += 1
        else:
```

```
            merged.append(right[j])
            j += 1

    merged.extend(left[i:])
    merged.extend(right[j:])
    return merged

def binary_search_flight(records_sorted, flight_id):
    left = 0
    right = len(records_sorted) - 1

    while left <= right:
        mid = (left + right) // 2
        current_id = records_sorted[mid]["flight_id"]

        if current_id == flight_id:
            return records_sorted[mid]
        if current_id < flight_id:
            left = mid + 1
        else:
            right = mid - 1

    return None

if __name__ == "__main__":
    flights = [
        {"flight_id": 802, "airline": "AirOne", "departure": "09:45", "arrival": "11:20",
"status": "On Time"},
```

```
    {"flight_id": 799, "airline": "SkyJet", "departure": "08:30", "arrival": "10:00", "status": "Delayed"},
    {"flight_id": 810, "airline": "CloudAir", "departure": "12:10", "arrival": "14:05", "status": "On Time"},
  ]

  by_id = merge_sort_records(flights, lambda x: x["flight_id"])
  by_departure = merge_sort_records(flights, lambda x: datetime.strptime(x["departure"], "%H:%M"))
  by_arrival = merge_sort_records(flights, lambda x: datetime.strptime(x["arrival"], "%H:%M"))

  print("Search flight ID 810:", binary_search_flight(by_id, 810))
  print("Sorted by departure time:", by_departure)
  print("Sorted by arrival time:", by_arrival)
```

Output

```
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding> & C:\Users\sidda\AppData\L
ocal\Programs\Python\Python314\python.exe "c:/Users/sidda/OneDrive/Documents/Desktop/AI - Ass
isted coding/.github/12.5.PY/task8_airport_management.py"
Search flight ID 810: {'flight_id': 810, 'airline': 'CloudAir', 'departure': '12:10', 'arriva
l': '14:05', 'status': 'On Time'}
Sorted by departure time: [{'flight_id': 799, 'airline': 'SkyJet', 'departure': '08:30', 'arr
ival': '10:00', 'status': 'Delayed'}, {'flight_id': 802, 'airline': 'AirOne', 'departure': '0
9:45', 'arrival': '11:20', 'status': 'On Time'}, {'flight_id': 810, 'airline': 'CloudAir', 'd
eparture': '12:10', 'arrival': '14:05', 'status': 'On Time'}]
Sorted by arrival time: [{'flight_id': 799, 'airline': 'SkyJet', 'departure': '08:30', 'arriv
al': '10:00', 'status': 'Delayed'}, {'flight_id': 802, 'airline': 'AirOne', 'departure': '09:
45', 'arrival': '11:20', 'status': 'On Time'}, {'flight_id': 810, 'airline': 'CloudAir', 'dep
arture': '12:10', 'arrival': '14:05', 'status': 'On Time'}]
PS C:\Users\sidda\OneDrive\Documents\Desktop\AI - Assisted coding>
```

powershell
Python
Python